이진 분류 성능평가 지표와 ROC-AUC

공식 · 해석 · Base R 구현 · 실행 예제 | 별도 R 패키지 불필요

이 자료는 이진 분류(binary classification)를 대상으로 합니다. 실제 레이블과 예측 레이블만 있으면 혼동행렬 기반 지표를 계산할 수 있습니다. ROC 곡선과 AUC에는 예측된 0/1 클래스 대신 연속형 예측 점수(또는 positive class 확률)가 추가로 필요합니다.

1. Confusion matrix와 기호

실제 / 예측	예측 Negative (0)	예측 Positive (1)
실제 Negative (0)	TN (True Negative)	FP (False Positive)
실제 Positive (1)	FN (False Negative)	TP (True Positive)

이 표는 행 = 실제(True), 열 = 예측(Predicted)입니다. R의 table(y_true, y_pred) 및
Python의 confusion_matrix(y_true, y_pred) 기본 출력과 같은 방향입니다.

P = TP + FN (실제 positive 수), N = TN + FP (실제 negative 수), n = P + N. 아래에서 분모가 0이면 해당 지표는 정의되지 않습니다.

2. 주요 분류 성능지표: 공식과 해석

지표	공식	의미
Accuracy	(TP + TN) / n	전체 관측치 중 올바르게 예측한 비율
Precision (PPV)	TP / (TP + FP)	Positive로 예측한 것 중 실제 positive 비율
Recall = Sensitivity = TPR	TP / (TP + FN)	실제 positive 중 positive로 탐지한 비율
Specificity = TNR	TN / (TN + FP)	실제 negative 중 negative로 탐지한 비율
FPR = 1 − Specificity	FP / (FP + TN)	실제 negative를 positive로 잘못 예측한 비율
F1-score	2PR / (P + R) = 2TP / (2TP + FP + FN)	Precision과 Recall의 조화평균
Balanced Accuracy	(Sensitivity + Specificity) / 2	두 클래스의 재현율을 동일 가중으로 평균
NPV	TN / (TN + FN)	Negative 예측 중 실제 negative 비율
FNR	FN / (FN + TP)	실제 positive를 놓친 비율

주의: 여기서 F1의 P와 R은 각각 Precision과 Recall을 의미합니다. Accuracy는 심한 클래스 불균형 상황에서 다수 클래스에 치우칠 수 있습니다. F1은 TN을 직접 반영하지 않으므로 연구 목적에 맞게 선택해야 합니다.

3. ROC curve와 AUC

양성 점수가 클수록 positive일 가능성이 높다고 가정합니다. 임계값 c에서 예측 규칙은 “score ≥ c이면 1, 아니면 0”입니다. c를 매우 큰 값에서 낮은 값으로 이동시키면서 FPR(c) = 1 − Specificity(c)를 가로축, TPR(c) = Sensitivity(c)를 세로축에 표시한 것이 ROC curve입니다.

AUC는 ROC 곡선 아래 면적이며, (인접 ROC 점을 직선으로 연결하는) 사다리꼴 공식으로 계산할 수 있습니다. 같은 점수가 있는 관측치는 한 번에 처리합니다. AUC는 임의의 positive가 임의의 negative보다 높은 점수를 받을 확률에 동점의 절반을 더한 값과 같습니다.

AUC = Pr(score_positive > score_negative) + 0.5 × Pr(score_positive = score_negative)

ROC-AUC는 순위 판별 능력을 평가합니다. 실제 적용할 threshold에서의 Accuracy·F1·기대 비용과는 다른 개념이며, 예측 확률의 보정(calibration)도 직접 평가하지 않습니다.

4. Base R 함수 ①: 혼동행렬 기반 지표

```
classification_metrics <- function(y_observed, y_predicted,
                                   positive = 1) {
  if (length(y_observed) != length(y_predicted) ||
      length(y_observed) == 0L)
    stop("Inputs must have equal, nonzero length.")
  if (anyNA(y_observed) || anyNA(y_predicted))
    stop("Missing values are not allowed.")

  observed_positive  <- y_observed == positive
  predicted_positive <- y_predicted == positive
  tp <- sum( observed_positive &  predicted_positive)
  tn <- sum(!observed_positive & !predicted_positive)
  fp <- sum(!observed_positive &  predicted_positive)
  fn <- sum( observed_positive & !predicted_positive)

  safe_div <- function(a, b) if (b == 0) NA_real_ else a / b
  precision   <- safe_div(tp, tp + fp)
  sensitivity <- safe_div(tp, tp + fn)
  specificity <- safe_div(tn, tn + fp)

  metrics <- c(
    Accuracy          = safe_div(tp + tn, tp + tn + fp + fn),
    Precision         = precision,
    Recall            = sensitivity,
    Sensitivity       = sensitivity,
    Specificity       = specificity,
    FPR               = safe_div(fp, fp + tn),
    FNR               = safe_div(fn, fn + tp),
    F1                = safe_div(2 * tp, 2 * tp + fp + fn),
    Balanced_Accuracy = (sensitivity + specificity) / 2,
    NPV               = safe_div(tn, tn + fn)
  )
  list(confusion = matrix(c(tp, fp, fn, tn), nrow = 2,
       dimnames = list(Predicted = c("Positive", "Negative"),
                       Actual = c("Positive", "Negative"))),
       metrics = metrics)
}
```

반환하는 confusion matrix도 행 = 예측, 열 = 실제입니다. 함수의 `positive`는 positive class를 지정합니다. 0/1 값뿐 아니라 문자형·factor 레이블도 입력할 수 있습니다. 반환값은 `$confusion` 및 `$metrics`입니다.

5. Base R 함수 ②: ROC curve + AUC

```
roc_auc <- function(y_observed, y_score,
                    positive = 1, draw = TRUE) {
  if (length(y_observed) != length(y_score) ||
      length(y_observed) == 0L)
    stop("Inputs must have equal, nonzero length.")
  if (anyNA(y_observed) || anyNA(y_score) ||
      !is.numeric(y_score) || any(!is.finite(y_score)))
    stop("Scores must be finite numeric values without NA.")

  is_pos <- y_observed == positive
  n_pos <- sum(is_pos)
  n_neg <- sum(!is_pos)
  if (n_pos == 0 || n_neg == 0)
    stop("Both classes must be present.")

  ord <- order(y_score, decreasing = TRUE)
  scores <- y_score[ord]
  is_pos <- is_pos[ord]
  tp <- cumsum(is_pos)
  fp <- cumsum(!is_pos)

  # Keep the last observation at each unique score.
  idx <- which(!duplicated(scores, fromLast = TRUE))
  tpr <- c(0, tp[idx] / n_pos)
  fpr <- c(0, fp[idx] / n_neg)
  thresholds <- c(Inf, scores[idx])

  auc <- sum(diff(fpr) *
             (head(tpr, -1) + tail(tpr, -1)) / 2)
  roc_points <- data.frame(
    threshold = thresholds,
    FPR = fpr, TPR = tpr,
    Specificity = 1 - fpr, Sensitivity = tpr)

  if (draw) {
    plot(fpr, tpr, type = "l", lwd = 2, col = "#2364AA",
         xlim = c(0, 1), ylim = c(0, 1),
         xlab = "1 - Specificity (FPR)",
         ylab = "Sensitivity (TPR)",
         main = sprintf("ROC curve (AUC = %.3f)", auc))
    abline(0, 1, col = "gray60", lty = 2)
  }
  list(AUC = auc, ROC = roc_points)
}
```

각 고유한 점수를 threshold로 사용합니다. 첫 점 (0, 0)은 어느 관측치도 positive로 예측하지 않는 경우입니다. `type = "l"`은 동점까지 처리한 ROC 점들을 선형 연결하며, 사다리꼴 AUC와 일치합니다.

6. 재현 가능한 실행 예제

```
y_observed <- c(1, 0, 1, 0, 1, 0, 1, 0, 1, 0)
y_score    <- c(.92, .70, .85, .30, .65,
                .45, .78, .20, .55, .10)
y_predicted <- as.integer(y_score >= 0.5)

result_cls <- classification_metrics(y_observed, y_predicted)
print(result_cls$confusion)
round(result_cls$metrics, 4)

result_roc <- roc_auc(y_observed, y_score)
result_roc$AUC
result_roc$ROC

# Rank-based AUC verification (Mann-Whitney identity)
pos <- y_observed == 1
n1 <- sum(pos); n0 <- sum(!pos)
auc_rank <- (sum(rank(y_score)[pos]) - n1*(n1+1)/2) / (n1*n0)
all.equal(result_roc$AUC, as.numeric(auc_rank))
```

예제 confusion matrix: 예측 Positive 행은 [TP = 5, FP = 1], 예측 Negative 행은 [FN = 0, TN = 4]입니다.

예제 결과: TP = 5, TN = 4, FP = 1, FN = 0, Accuracy = 0.9000, Precision = 0.8333, Recall = 1.0000, Specificity = 0.8000, F1 = 0.9091, Balanced Accuracy = 0.9000, AUC = 0.9200.

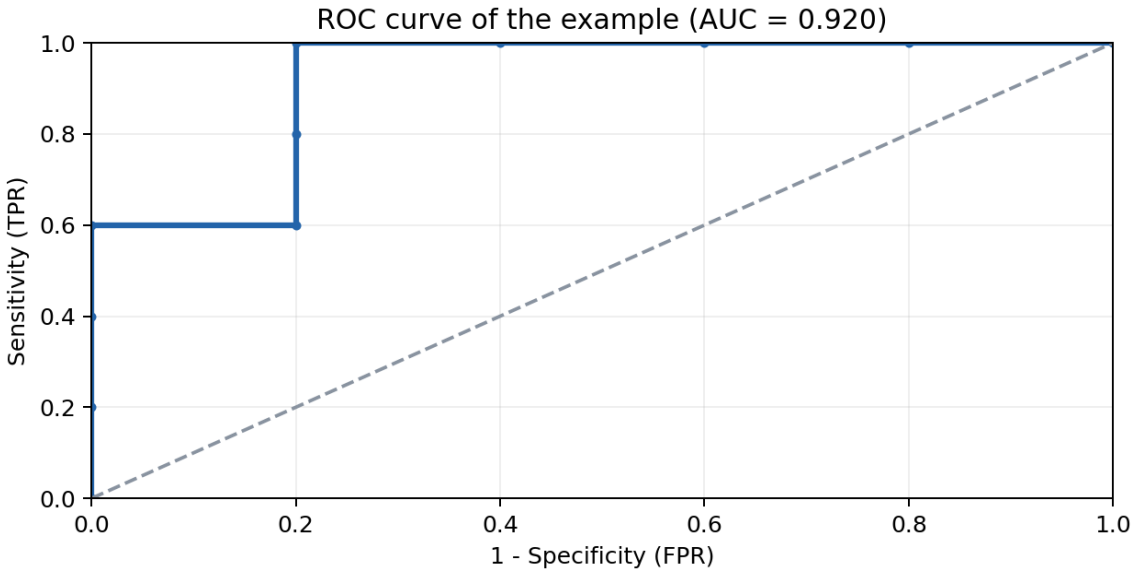


그림: 10개 관측치에서 threshold를 낮추며 얻은 경험적 ROC 좌표를 직선으로 연결한 결과.

7. 해석 및 실무상 유의점

① Accuracy, F1, Sensitivity, Specificity 등은 선택한 threshold에 의존합니다. 반면 ROC-AUC는 threshold를 변화시키며 평가한 순위 성능입니다.

② F1은 precision과 recall을 조화평균하지만 negative 예측이 올바른지(TN)는 직접 포함하지 않습니다. 불균형 자료에서 하나의 지표만으로 성능을 판단하지 않는 편이 좋습니다.

③ 오분류 비용이 명확하다면 FP와 FN의 비용을 반영한 기대 손실을 평가하는 것이 의사결정 목적에 직접 부합합니다. 오분류 비용이 불확실할 때는 여러 비용비율에 대한 민감도 분석이 도움이 됩니다.

④ 모델이나 threshold를 선택하는 데 사용한 자료와 최종 성능을 보고하는 test 자료는 구분해야 합니다. 최종 ROC-AUC 역시 가능하면 독립적인 test 자료에서 계산합니다.

⑤ 이 자료의 함수는 binary classification에 한정됩니다. Multiclass의 macro/micro averaging은 별도의 정의가 필요합니다.